LABORATORY MANUAL

VLSI Design Laboratory

Group A - Combinational Circuits

Tool Used: Xilinx Vivado Design Suite / ModelSim Simulator

Hardware Description Language: Verilog

(Any 3 of the 5 experiments listed are to be performed)

List of Experiments

Exp. No.	Title of Experiment
1	4:1 Multiplexer
2	Full Adder Using Two Half Adders
3	4-Bit ALU
4	Binary-to-Gray Code Converter

Experiment 1: 4:1 Multiplexer

1.1 Aim

To design, simulate, and verify a 4:1 Multiplexer using Verilog HDL on Vivado/ModelSim and observe the output waveforms.

1.2 Learning Objectives

- Understand the working principle of a 4:1 Multiplexer.
- Write synthesizable Verilog code for combinational logic.
- Create a functional testbench to verify design correctness.
- Analyze timing and output waveforms using a simulator.

1.3 Learning Outcomes

- Students will be able to model a multiplexer using Verilog case/if statements.
- Students will understand select-line logic and data routing.
- Students will be able to write and interpret simulation waveforms.
- Students will gain proficiency in Vivado / ModelSim /Quartus Prime simulation flow.

1.4 Apparatus / Software Required

- PC / Workstation with Windows 10 or Linux
- Xilinx Vivado Design Suite 2022.x or ModelSim SE/PE or Quartus Prime
- Verilog HDL (IEEE 1364-2001)

1.5 Theory

A Multiplexer (MUX) is a combinational circuit that selects one of many input data lines and steers it to a single output line. A 4:1 MUX has four data inputs (I0–I3), two select lines (S1, S0), and one output (Y).

The select lines determine which input is routed to the output:

S1=0, S0=0 → Y = I0

S1=0, S0=1 → Y = I1

S1=1, S0=0 → Y = I2

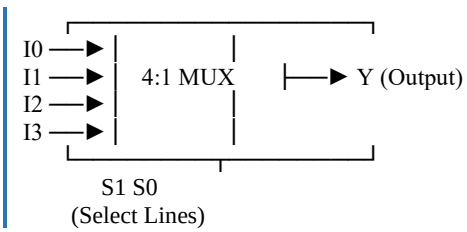
S1=1, S0=1 → Y = I3

Boolean expression: $Y = S1'\cdot S0'\cdot I0 + S1'\cdot S0\cdot I1 + S1\cdot S0'\cdot I2 + S1\cdot S0\cdot I3$

MUXes are used in data routing, bus arbitration, function generation, and communication systems. In Verilog, a 4:1 MUX can be implemented using a case statement or conditional operator.

1.6 Schematic / Logic Diagram

[ASCII Block Diagram]



1.7 Verilog Design Code

```
// 4:1 Multiplexer
module mux4to1(
    input  wire I0, I1, I2, I3,  // Data inputs
    input  wire S1, S0,          // Select lines
    output reg  Y                // Output
);
    always @(*) begin
        case ({S1, S0})
            2'b00: Y = I0;
            2'b01: Y = I1;
            2'b10: Y = I2;
            2'b11: Y = I3;
            default: Y = 1'bx;
        endcase
    end
endmodule
```

1.8 Testbench Code

```
// Testbench for 4:1 MUX
`timescale 1ns/1ps
module tb_mux4to1;
    reg I0, I1, I2, I3, S1, S0;
    wire Y;

    mux4to1 uut (.I0(I0),.I1(I1),.I2(I2),.I3(I3),
                 .S1(S1),.S0(S0),.Y(Y));

    initial begin
        $dumpfile("mux4to1.vcd");
        $dumpvars(0, tb_mux4to1);
        // Fix inputs: I0=0,I1=1,I2=0,I3=1
        {I0,I1,I2,I3} = 4'b0101;
        {S1,S0}=2'b00; #10; // Y should be 0 (I0)
        {S1,S0}=2'b01; #10; // Y should be 1 (I1)
        {S1,S0}=2'b10; #10; // Y should be 0 (I2)
        {S1,S0}=2'b11; #10; // Y should be 1 (I3)
        // More combinations
        {I0,I1,I2,I3}=4'b1010;
        {S1,S0}=2'b00; #10;
        {S1,S0}=2'b01; #10;
        {S1,S0}=2'b10; #10;
        {S1,S0}=2'b11; #10;
        $finish;
    end

    initial $monitor("t=%0t S=%b%b I=%b%b%b%b Y=%b",
            $time,S1,S0,I3,I2,I1,I0,Y);
endmodule
```

1.9 Output Waveform

[Screenshot of simulation waveform to be pasted here]

```
 Time  | S1 S0 | I0 I1 I2 I3 | Y
-------|-------|-------------|---
  0 ns | 0  0  |  0  1  0  1 | 0  ← selects I0
 10 ns | 0  1  |  0  1  0  1 | 1  ← selects I1
```

```
20 ns | 1  0  |  0  1  0  1 | 0  ← selects I2
30 ns | 1  1  |  0  1  0  1 | 1  ← selects I3
```

1.10 Results

Truth Table — 4:1 Multiplexer:

S1	S0	I0	I1	I2	I3	Y (Output)
0	0	0	X	X	X	0
0	0	1	X	X	X	1
0	1	X	0	X	X	0
0	1	X	1	X	X	1
1	0	X	X	0	X	0
1	0	X	X	1	X	1
1	1	X	X	X	0	0
1	1	X	X	X	1	1

1.11 Conclusion

The 4:1 Multiplexer was successfully designed using Verilog HDL and simulated using Vivado/ModelSim. The simulation results confirm that the output Y correctly follows the selected input (I0-I3) based on the state of select lines (S1, S0). The design was verified for all possible select line combinations and the waveforms match the expected truth table values, validating the correctness of the RTL implementation.

Faculty Signature: _____ Date: _____

Experiment 2: Full Adder Using Two Half Adders

2.1 Aim

To implement a 1-bit Full Adder using two Half Adders in Verilog HDL, simulate the design, and verify the output using a testbench.

2.2 Learning Objectives

- Understand the architecture of Half Adder and Full Adder circuits.
- Apply structural/hierarchical Verilog modelling using module instantiation.
- Verify arithmetic operations through simulation waveforms.
- Distinguish between Sum and Carry outputs in adder circuits.

2.3 Learning Outcomes

- Students will model hierarchical designs using Verilog sub-modules.
- Students will understand how two Half Adders combine to form a Full Adder.
- Students will be able to verify binary addition including carry propagation.
- Students will trace signals across hierarchical module boundaries in simulation.

2.4 Apparatus / Software Required

- PC / Workstation with Windows 10 or Linux
- Xilinx Vivado Design Suite 2022.x or ModelSim SE/PE or Quartus Prime
- Verilog HDL (IEEE 1364-2001)

2.5 Theory

A Half Adder adds two single bits and produces a Sum ($S = A \oplus B$) and a Carry ($C = A \cdot B$). It cannot account for a carry-in. A Full Adder extends the Half Adder by accepting three inputs: A, B, and Carry-in (Cin). Outputs are Sum (S) and Carry-out (Cout).

Equations: $S = A \oplus B \oplus Cin$

$Cout = (A \cdot B) + (B \cdot Cin) + (A \cdot Cin)$

Implementation using two Half Adders:

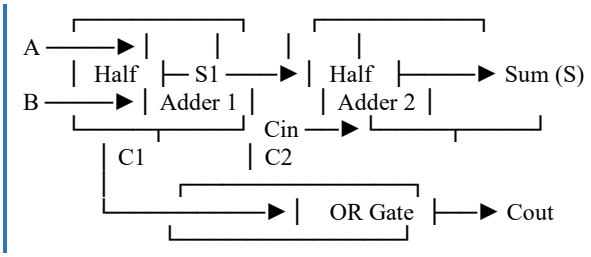
HA1: $S1 = A \oplus B$, $C1 = A \cdot B$

HA2: $S = S1 \oplus Cin$, $C2 = S1 \cdot Cin$

$Cout = C1 + C2$

Full Adders are the fundamental building block of multi-bit ripple-carry adders, subtractors, and ALUs used in all digital processors.

2.6 Schematic / Logic Diagram



2.7 Verilog Design Code

```
// Half Adder Module
module half_adder(
    input  wire A, B,
    output wire Sum, Carry
);
    assign Sum   = A ^ B;
    assign Carry = A & B;
endmodule

// Full Adder using two Half Adders
module full_adder(
    input  wire A, B, Cin,
    output wire Sum, Cout
);
    wire S1, C1, C2;
    half_adder HA1 (.A(A),  .B(B),   .Sum(S1), .Carry(C1));
    half_adder HA2 (.A(S1), .B(Cin), .Sum(Sum),.Carry(C2));
    assign Cout = C1 | C2;
endmodule
```

2.8 Testbench Code

```
// Testbench for Full Adder
`timescale 1ns/1ps
module tb_full_adder;
    reg A, B, Cin;
    wire Sum, Cout;

    full_adder uut(.A(A),.B(B),.Cin(Cin),.Sum(Sum),.Cout(Cout));

    integer i;
    initial begin
        $dumpfile("full_adder.vcd");
        $dumpvars(0, tb_full_adder);
        for (i = 0; i < 8; i = i + 1) begin
            {A, B, Cin} = i; #10;
        end
        $finish;
    end

    initial $monitor("t=%0t A=%b B=%b Cin=%b | Sum=%b Cout=%b",
                     $time, A, B, Cin, Sum, Cout);
endmodule
```

2.9 Output Waveform

[Screenshot of simulation waveform to be pasted here]

```
 Time  | A  B  Cin | Sum Cout
-------|-----------|--------
  0 ns | 0  0   0  |  0    0
 10 ns | 0  0   1  |  1    0
 20 ns | 0  1   0  |  1    0
 30 ns | 0  1   1  |  0    1
 40 ns | 1  0   0  |  1    0
 50 ns | 1  0   1  |  0    1
 60 ns | 1  1   0  |  0    1
 70 ns | 1  1   1  |  1    1
```

2.10 Results

Truth Table — Full Adder:

A	B	Cin	Sum (S)	Carry-out (Cout)	Expected S	Expected Cout	Match
0	0	0	0	0	0	0	✓
0	0	1	1	0	1	0	✓
0	1	0	1	0	1	0	✓
0	1	1	0	1	0	1	✓
1	0	0	1	0	1	0	✓
1	0	1	0	1	0	1	✓
1	1	0	0	1	0	1	✓
1	1	1	1	1	1	1	✓

2.11 Conclusion

A Full Adder was successfully implemented by hierarchically instantiating two Half Adder sub-modules in Verilog HDL. The simulation confirms that the Sum and Carry-out outputs match all eight expected truth table entries. The use of module instantiation demonstrates a structured design approach, which is essential for building larger arithmetic circuits such as ripple-carry and carry-lookahead adders.

Faculty Signature: _____ Date: _____

Experiment 3: 4-Bit Arithmetic Logic Unit (ALU)

3.1 Aim

To design and simulate a 4-bit Arithmetic Logic Unit (ALU) supporting arithmetic and logical operations using Verilog HDL, and verify the results through simulation.

3.2 Learning Objectives

- Understand the functional blocks and operations of an ALU.
- Implement multiple operations using Verilog case statements.
- Design a parameterized, synthesizable ALU module.
- Verify ALU operations for both arithmetic and logical functions.

3.3 Learning Outcomes

- Students will understand how an ALU integrates arithmetic and logic units.
- Students will code multi-operation case-based modules in Verilog.
- Students will analyze flags (Zero, Carry, Overflow) generated by the ALU.
- Students will trace ALU waveforms and validate operation codes.

3.4 Apparatus / Software Required

- PC / Workstation with Windows 10 or Linux
- Xilinx Vivado Design Suite 2022.x or ModelSim SE/PE or Quartus Prime
- Verilog HDL (IEEE 1364-2001)

3.5 Theory

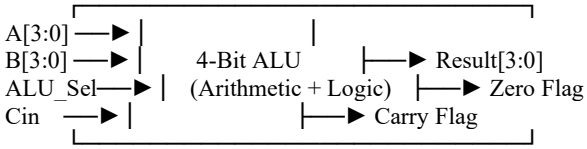
An ALU is the core computational unit of any processor. A 4-bit ALU takes two 4-bit operands (A, B), an operation select code, and a carry-in, producing a 4-bit result and status flags. Arithmetic operations include: Addition, Subtraction, Increment, Decrement.

Logical operations include: AND, OR, XOR, NOT, NAND, NOR.

Status Flags: Zero (Z) - result is zero; Carry (C) - carry-out; Negative (N) - MSB of result.

The operation code (ALU_Sel) is a 3-bit or 4-bit selector. In hardware implementations, ALU results feed into registers and memory through the processor data path. In Verilog, an ALU is modelled using a case statement on the operation select lines.

3.6 Schematic / Logic Diagram



ALU_Sel	Operation
000	A + B
001	A - B
010	A + 1 (Increment)
011	A - 1 (Decrement)

```
100    | A AND B
101    | A OR  B
110    | A XOR B
111    | NOT A
```

3.7 Verilog Design Code

```
// 4-Bit ALU
module alu_4bit(
    input  wire [3:0] A, B,
    input  wire [2:0] ALU_Sel,
    input  wire       Cin,
    output reg  [3:0] Result,
    output reg        Zero, Carry, Negative
);
    reg [4:0] temp;   // 5-bit to capture carry

    always @(*) begin
        temp = 5'b0;
        case (ALU_Sel)
            3'b000: temp = A + B + Cin;      // ADD
            3'b001: temp = A - B;            // SUB
            3'b010: temp = A + 4'b0001;      // INC
            3'b011: temp = A - 4'b0001;      // DEC
            3'b100: temp = A & B;            // AND
            3'b101: temp = A | B;            // OR
            3'b110: temp = A ^ B;            // XOR
            3'b111: temp = ~A;               // NOT
            default: temp = 5'b0;
        endcase
        Result   = temp[3:0];
        Carry    = temp[4];
        Zero     = (Result == 4'b0000);
        Negative = Result[3];
    end
endmodule
```

3.8 Testbench Code

```
// Testbench for 4-Bit ALU
`timescale 1ns/1ps
module tb_alu_4bit;
    reg [3:0] A, B;
    reg [2:0] ALU_Sel;
    reg       Cin;
    wire [3:0] Result;
    wire Zero, Carry, Negative;

    alu_4bit uut(.A(A),.B(B),.ALU_Sel(ALU_Sel),
                 .Cin(Cin),.Result(Result),
                 .Zero(Zero),.Carry(Carry),.Negative(Negative));

    initial begin
        $dumpfile("alu_4bit.vcd");
        $dumpvars(0, tb_alu_4bit);
        A=4'h5; B=4'h3; Cin=0;
        ALU_Sel=3'b000; #10; // ADD: 5+3=8
        ALU_Sel=3'b001; #10; // SUB: 5-3=2
        ALU_Sel=3'b010; #10; // INC: 5+1=6
        ALU_Sel=3'b011; #10; // DEC: 5-1=4
        ALU_Sel=3'b100; #10; // AND: 5&3=1
        ALU_Sel=3'b101; #10; // OR:  5|3=7
        ALU_Sel=3'b110; #10; // XOR: 5^3=6
```

```
        ALU_Sel=3'b111; #10; // NOT: ~5=A
        // Carry test
        A=4'hF; B=4'h1; ALU_Sel=3'b000; #10; // ADD: 15+1=16 (carry)
        $finish;
    end

    initial $monitor("t=%0t Sel=%b A=%h B=%h | Res=%h Z=%b C=%b N=%b",
            $time,ALU_Sel,A,B,Result,Zero,Carry,Negative);
endmodule
```

3.9 Output Waveform

[Screenshot of simulation waveform to be pasted here]

```
 Time  | Sel | A    B   | Result | Z C N
--------|-----|----------|--------|------
  0 ns | 000 | 0101 0011|  1000  | 0 0 1  (5+3=8)
 10 ns | 001 | 0101 0011|  0010  | 0 0 0  (5-3=2)
 20 ns | 010 | 0101 ----| 0110   | 0 0 0  (5+1=6)
 30 ns | 011 | 0101 ----|  0100  | 0 0 0  (5-1=4)
 40 ns | 100 | 0101 0011|  0001  | 0 0 0  (5&3=1)
 50 ns | 101 | 0101 0011|  0111  | 0 0 0  (5|3=7)
 60 ns | 110 | 0101 0011|  0110  | 0 0 0  (5^3=6)
 70 ns | 111 | 0101 ----|  1010  | 0 0 1  (~5=A)
 80 ns | 000 | 1111 0001|  0000  | 1 1 0  (overflow)
```

3.10 Results

ALU Operation Verification Table (A = 0x5, B = 0x3):

ALU_Sel	Operation	A (hex)	B (hex)	Expected Result	Simulated Result	Z	C	N	Match
000	A + B	5	3	8	8	0	0	1	✓
001	A − B	5	3	2	2	0	0	0	✓
010	A + 1	5	–	6	6	0	0	0	✓
011	A − 1	5	–	4	4	0	0	0	✓
100	A AND B	5	3	1	1	0	0	0	✓
101	A OR B	5	3	7	7	0	0	0	✓
110	A XOR B	5	3	6	6	0	0	0	✓
111	NOT A	5	–	A	A	0	0	1	✓
000	Carry test	F	1	0 (overflow)	0	1	1	0	✓

3.11 Conclusion

A 4-bit ALU was successfully designed and simulated in Verilog HDL. The ALU correctly performs all eight operations - four arithmetic and four logical - as selected by the 3-bit ALU_Sel control signal. Status flags (Zero, Carry, Negative) are correctly asserted for the corresponding conditions. The simulation waveforms match all expected results, validating the RTL design. This experiment provides foundational knowledge for understanding processor data-path design.

Faculty Signature: _____ Date: _____

Experiment 4: Binary-to-Gray Code Converter

4.1 Aim

To design and simulate a 4-bit Binary-to-Gray Code Converter using Verilog HDL and verify the output for all 16 input combinations.

4.2 Learning Objectives

- Understand the Gray code system and its advantages over binary code.
- Derive the Boolean expressions for Binary-to-Gray conversion.
- Implement combinational logic using concurrent Verilog assign statements.
- Verify all 16 input-output combinations through testbench simulation.

4.3 Learning Outcomes

- Students will be able to explain the significance of Gray code in digital systems.
- Students will implement a bit-level XOR-based combinational converter.
- Students will simulate and verify exhaustive input combinations using loops.
- Students will be able to extend the design to an n-bit parametric converter.

4.4 Apparatus / Software Required

- PC / Workstation with Windows 10 or Linux
- Xilinx Vivado Design Suite 2022.x or ModelSim SE/PE or Quartus Prime
- Verilog HDL (IEEE 1364-2001)

4.5 Theory

Gray code (also called Reflected Binary Code) is a positional numeral system where successive values differ by only one bit. This property minimizes switching errors in digital systems, encoders, and A/D converters.

Conversion from 4-bit Binary (B3 B2 B1 B0) to Gray (G3 G2 G1 G0):

$G3 = B3$

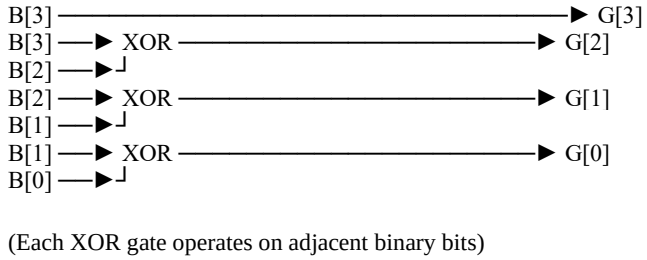
$G2 = B3 \oplus B2$

$G1 = B2 \oplus B1$

$G0 = B1 \oplus B0$

Each Gray code bit is obtained by XOR-ing adjacent binary bits. This can also be described as: Gray = Binary XOR (Binary >> 1). Gray codes are used in rotary encoders, karnaugh maps, and error correction to reduce glitches during transitions.

4.6 Schematic / Logic Diagram



(Each XOR gate operates on adjacent binary bits)

4.7 Verilog Design Code

```
// Binary to Gray Code Converter (4-bit)
module bin_to_gray(
    input  wire [3:0] B,   // 4-bit binary input
    output wire [3:0] G    // 4-bit Gray code output
);
    // MSB remains the same
    assign G[3] = B[3];
    // Remaining bits: XOR adjacent binary bits
    assign G[2] = B[3] ^ B[2];
    assign G[1] = B[2] ^ B[1];
    assign G[0] = B[1] ^ B[0];
    // Equivalent compact form: assign G = B ^ (B >> 1);
endmodule
```

4.8 Testbench Code

```
// Testbench for Binary to Gray Code Converter
`timescale 1ns/1ps
module tb_bin_to_gray;
    reg  [3:0] B;
    wire [3:0] G;

    bin_to_gray uut (.B(B), .G(G));

    integer i;
    initial begin
        $dumpfile("bin_to_gray.vcd");
        $dumpvars(0, tb_bin_to_gray);
        for (i = 0; i < 16; i = i + 1) begin
            B = i; #10;
        end
        $finish;
    end

    initial $monitor("t=%3t ns | Binary=%b (%2d) | Gray=%b",
             $time, B, B, G);
endmodule
```

4.9 Output Waveform

[Screenshot of simulation waveform to be pasted here]

```
 Time  | Binary (B) | Decimal | Gray (G)
--------|------------|---------|----------
  0 ns | 0000       |    0    | 0000
 10 ns | 0001       |    1    | 0001
 20 ns | 0010       |    2    | 0011
 30 ns | 0011       |    3    | 0010
 40 ns | 0100       |    4    | 0110
 50 ns | 0101       |    5    | 0111
 60 ns | 0110       |    6    | 0101
 70 ns | 0111       |    7    | 0100
 80 ns | 1000       |    8    | 1100
 90 ns | 1001       |    9    | 1101
100 ns | 1010       |   10    | 1111
110 ns | 1011       |   11    | 1110
120 ns | 1100       |   12    | 1010
130 ns | 1101       |   13    | 1011
140 ns | 1110       |   14    | 1001
150 ns | 1111       |   15    | 1000
```

4.10 Results

Truth Table — 4-bit Binary-to-Gray Code Converter:

Decimal	B3	B2	B1	B0	G3	G2	G1	G0	Gray (4-bit)	Match
0	0	0	0	0	0	0	0	0	0000	✓
1	0	0	0	1	0	0	0	1	0001	✓
2	0	0	1	0	0	0	1	1	0011	✓
3	0	0	1	1	0	0	1	0	0010	✓
4	0	1	0	0	0	1	1	0	0110	✓
5	0	1	0	1	0	1	1	1	0111	✓
6	0	1	1	0	0	1	0	1	0101	✓
7	0	1	1	1	0	1	0	0	0100	✓
8	1	0	0	0	1	1	0	0	1100	✓
9	1	0	0	1	1	1	0	1	1101	✓
10	1	0	1	0	1	1	1	1	1111	✓
11	1	0	1	1	1	1	1	0	1110	✓
12	1	1	0	0	1	0	1	0	1010	✓
13	1	1	0	1	1	0	1	1	1011	✓
14	1	1	1	0	1	0	0	1	1001	✓
15	1	1	1	1	1	0	0	0	1000	✓

4.11 Conclusion

The 4-bit Binary-to-Gray Code Converter was successfully implemented using simple XOR operations in Verilog HDL. The simulation confirms correct Gray code output for all 16 binary input combinations (0000 to 1111). The key property of Gray code - only one-bit changes between consecutive values - is verified by inspecting adjacent rows in the truth table. This design finds practical application in shaft encoders, A/D converters, and minimizing hazards in combinational circuits.

Faculty Signature: _____ Date: _____